

Метод отжига

24 марта 2026 г.

1. Общая информация

В настоящее время метод отжига применяется для решения многих оптимизационных задач — финансовых, компьютерной графики, комбинаторных в телекоммуникационных сетях, и многих других. Зачастую метод отжига используют для обучения нейронных сетей. Несмотря на такую широкую область применения, скорость сходимости метода отжига всё ещё мало изучена.

Огромным преимуществом метода отжига является свойство избегать «ловушки» в локальных минимумах оптимизируемой функции и продолжить поиск глобального минимума. Это достигается за счёт принятия не только изменений параметров, приводящих к уменьшению значения функции, но и некоторых изменений, увеличивающих её значение, в зависимости от т. н. температуры T характеристики моделируемого процесса. Чем выше температура, тем большие «ухудшающие» изменения (аналогичные случайным флуктуациям в нагретом веществе) допустимы, и больше их вероятность.

2. Описание алгоритма

Метод отжига служит для поиска глобального минимума некоторой функции $f(x)$, заданной на некотором пространстве S , дискретном или непрерывном. Элементы множества S представляют собой состояния физической системы, а значение функции f в этих точках используется как энергия системы $E = f(x)$.

В каждый момент предполагается заданной температура системы $T > 0$, как правило уменьшающаяся с течением времени. Находясь в состоянии x при температуре T , следующее состояние системы выбирается в соответствии с заданным порождающим семейством вероятностных распределений $G(x, T)$, которое при фиксированных x и T задаёт случайный элемент $x' = G(x, T)$ из S .

После генерации нового состояния $x' = G(x, T)$, система с вероятностью $h(\Delta E, T)$ переходит к следующему шагу в состояние x' , в противном случае процесс генерации x' повторяется.

$\Delta E = f(x') - f(x)$ обозначает приращение функции энергии. Величина $h(\Delta E, T)$ называется вероятностью принятия нового состояния.

Как правило, в качестве функции $h(\Delta E, T)$ выбирается либо точное значение соответствующей физической величины:

$$h(\Delta E, T) = \frac{1}{1 + \exp(\frac{\Delta E}{T})} \text{ (формула Ферми-Дирака)}$$

либо приближённое значение:

$$h(\Delta E, T) = \exp(\frac{-\Delta E}{T}).$$

Вторая формула используется наиболее часто. При её использовании $h(\Delta E, T)$ оказывается больше единицы в случае $\Delta E < 0$, и тогда соответствующая вероятность считается равной 1. Таким образом, если новое состояние даёт лучшее значение оптимизируемой функции, то переход в это состояние произойдёт в любом случае.

Общая схема:

1. Случайным образом выбирается начальная точка $x = x_0, x_0 \in S$. Текущее значение энергии E устанавливается в значение $f(x_0)$.
2. k -я итерация основного цикла состоит из следующих шагов:
 - (a) Сравнить энергию системы E в состоянии x с найденным на текущий момент глобальным минимумом. Если $E = f(x)$ меньше, то изменить значение глобального минимума.
 - (b) Сгенерировать новую точку $x' = G(x, T(k))$.

- (с) Вычислить значение функции в ней $E' = f(x')$.
- (d) Сгенерировать случайное число α из интервала $[0; 1]$.
- (е) Если $\alpha < h(E' - E, T(k))$, то установить $x \leftarrow x'$, $E \leftarrow E'$ и перейти к следующей итерации. Иначе повторить шаг (b), пока не будет найдена подходящая точка x' .

Известны следующие модификации этого алгоритма (вышеописанный метод отжига называется **гомогенным**):

- **Модификация А.** На шаге 2.(е) переход к следующей итерации происходит и в том случае, если точка x' не являлась подходящей (то есть меняем температуру в любом случае). При этом следующая итерация начинается с точки x , но уже с новым значением температуры.
- **Модификация Б.** В качестве оценки точки минимума возвращается последнее значение. Это может незначительно ускорить алгоритм в случае большой размерности S , но с небольшой вероятностью может привести к тому, что будет получено худшее решение (особенно, если температура к моменту завершения алгоритма остаётся значительно больше нуля). Эту модификацию практически не используют, поскольку память компьютера сильно подешевела.

3. Реализации

3.1. Больцмановский отжиг

Температура на k -ом состоянии системы задаётся формулой:

$$T(k) = \frac{T_0}{\ln(1 + k)}$$

Порождающее семейство вероятностных распределений $G(x, T)$ выбирается как семейство нормальных распределений с математическим ожиданием x и дисперсией T и соответственно задаётся плотностью (то есть ожидаемое значение $-x$, причем отклонение от центра всегда уменьшается):

$$g(x'; x, T) = (2\pi T)^{-\frac{n}{2}} e^{-\frac{|x-x'|^2}{2T}}$$

Причем пространство состояний предполагается метрическим с размерностью n (просто формула многомерного нормального распределения).

Функция G выдает состояние системы из множества S в соответствие с плотностью $g(x'; x, T)$ для каждого состояния системы x' . Работает это так: генерируется случайное число $a \in [0, 1]$ и ищется такое состояние x' , что $\int_{-\infty}^{x'} g(y; x, T) dy = a$ (то есть обратное действие).

Недостатком схемы больцмановского отжига является очень медленное уменьшение температуры T . Решение этой проблемы возможно путём замены закона изменения температуры, например, на следующий: $T(k) = rT(k-1)$, где $r \in (0, 7; 0, 9)$. Такая схема имитации отжига называется тушением. Она очень быстро сходится, что позволяет экономить вычислительные ресурсы. При этом не гарантируется нахождения глобального минимума, но, как правило, быстро находится близкое решение, а на практике и сам минимум.

Для Больцмановской схемы доказано, что при достаточно большом значении T_0 и общем количестве шагов k выбор такого семейства распределений гарантирует нахождение глобального минимума.

Доказывается это следующим образом ([полное доказательство](#) занимает 20 страниц):

Пусть Δx — это предлагаемое изменение параметра. Функция плотности нормального распределения с нулевым средним и дисперсией T выглядит так:

$$P(\Delta x) = \frac{1}{\sqrt{\pi T}} \exp\left(-\frac{\Delta x^2}{T}\right)$$

Предположим, наш алгоритм попал в локальный минимум. Чтобы из него выбраться, алгоритму нужно сгенерировать шаг длины $> L$. То есть нам нужно найти вероятность того, что модуль шага будет больше или равен L :

$$P(|\Delta x| \geq L) = 2 \int_L^\infty \frac{1}{\sqrt{\pi T}} \exp\left(-\frac{x^2}{T}\right) dx \quad (\text{в силу симметрии плотности})$$

Зная асимптотическое разложение:

$$\int_z^\infty e^{-t^2} dt \approx \frac{e^{-z^2}}{2z} \quad \text{при } z \rightarrow \infty$$

Сделаем замену $t = \frac{x}{\sqrt{T}}$ и применив асимптотику, получаем приближенное выражение для вероятности прыжка:

$$P(|\Delta x| \geq L) \approx \frac{\sqrt{T}}{L\sqrt{\pi}} \exp\left(-\frac{L^2}{T}\right)$$

Далее полезно доказать следующий факт:

Лемма Бореля — Кантелли:

Пусть $\{E_n\}$ — последовательность независимых событий в вероятностном пространстве. Пусть

$$E = \limsup_{n \rightarrow \infty} E_n = \bigcap_{N=1}^{\infty} \bigcup_{k=N}^{\infty} E_k$$

(в смысле индикаторов соответствующих событий, $G_N = \bigcup_{k=N}^{\infty} E_k$, тогда $I_{G_N}(\omega) = \sup_{k \geq N} I_{E_k}(\omega)$, $I_E(\omega) = \inf_k I_{G_k}(\omega) = \limsup_{n \rightarrow \infty} E_n$ в силу невозрастания последовательности $I_{G_N}(\omega)$).

Если сумма их вероятностей расходится: $\sum_{n=1}^{\infty} P(E_n) = \infty$, то вероятность того, что произойдет бесконечно много этих событий, равна 1: $P(\limsup_{n \rightarrow \infty} E_n) = 1$.

Доказательство:

$$E^c = \bigcup_{N=1}^{\infty} \bigcap_{k=N}^{\infty} E_k^c$$

Рассмотрим вероятность того, что события не происходят в интервале от N до M :

$$P\left(\bigcap_{k=N}^M E_k^c\right) = \prod_{k=N}^M P(E_k^c) = \prod_{k=N}^M (1 - P(E_k)) \leq \prod_{k=N}^M \exp(-P(E_k)) = \exp\left(-\sum_{k=N}^M P(E_k)\right)$$

При $M \rightarrow \infty$ сумма в экспоненте стремится к бесконечности:

$$\lim_{M \rightarrow \infty} \exp\left(-\sum_{k=N}^M P(E_k)\right) = e^{-\infty} = 0$$

Следовательно, для любого фиксированного N :

$$P\left(\bigcap_{k=N}^{\infty} E_k^c\right) = 0$$

Поскольку E^c — это счетное объединение множеств с нулевой вероятностью, то $P(E^c) = 0$. Значит, $P(E) = 1$.

Также несложно показать (что и происходит при быстром тушении $T(k) = r^k$), что если $\sum_{n=1}^{\infty} \mathbb{P}(E_n) < \infty$, то $\mathbb{P}(E) = 0$,
чтд.

Определим A_k как событие: на шаге k алгоритм предложил прыжок длиной $|\Delta x| \geq L$. Если ряд $P(A_k)$ будет сходиться, значит алгоритм предпримет лишь конечное количество попыток сделать шаг длины $> L$, то есть может застрять в локальном минимуме.

Далее можно показать, что для Гауссова порождающего семейства ряд вероятностей расходится (а значит, алгоритм не застрянет навсегда) только в том случае, если температура убывает не быстрее, чем логарифм: $T(k) \propto 1/\ln(k)$.

Однако на практике такое охлаждение неприменимо из-за колоссальных затрат времени: пусть $\exp(-\frac{L^2}{T}) = 10^{-3}$, тогда если температура упала в 2 раза, то вероятность прыжка уменьшилась в 1000 раз. То есть если мы попали в локальный минимум и температура уже достаточно мала, то вероятность выбраться из него очень мала.

3.2. Сверхбыстрый отжиг (VFSA)

Вследствие вышенаписанного становится понятно, что использовать нормальное распределение для выбора состояния - идея плохая. Возникает идея использовать другие распределения, например - распределение Коши. Будет введена новая функция распределения, у которой хвосты ещё тяжелее, нежели у Коши.

Будем считать, что пространство S состоит из D -мерных векторов $(x_1, x_2, \dots, x_D)$, где $x_i \in [A_i, B_i]$, причем температура T также является D -мерным вектором.

Пусть для конкретного состояния x' при текущем состоянии x $y_i = x'_i - x_i$, отнормируем эту величину поделив её на $B_i - A_i$. Семейство распределений вводится так: вводится функция

$$g_{(i;T)}(y_i) = \frac{1}{2(|y_i| + T_i) \ln(1 + \frac{1}{T_i})}, \text{ где } y_i \in [-1, 1].$$

Нетрудно убедиться, что эта функция действительно задаёт плотность некоторой случайной величины:

Пусть $K = \frac{1}{2 \ln(1+1/T_i)}$. Это константа, которая не зависит от y_i . Тогда плотность для одной координаты:

$$g_{(i;T)}(y_i) = K \cdot \frac{1}{|y_i| + T_i}$$

$$\int_{-1}^1 g_{(i;T)}(y_i) dy_i = K \int_{-1}^1 \frac{1}{|y_i| + T_i} dy_i$$

Так как функция $|y_i|$ симметрична относительно нуля, можно взять интеграл от 0 до 1 и умножить на 2:

$$2K \int_0^1 \frac{1}{y_i + T_i} dy_i$$

Первообразная функции $\frac{1}{y_i + T_i}$ — это $\ln(y_i + T_i)$.

$$2K \cdot [\ln(y_i + T_i)]_0^1 = 2K \cdot (\ln(1 + T_i) - \ln(T_i))$$

$$2K \cdot \ln\left(\frac{1 + T_i}{T_i}\right) = 2K \cdot \ln\left(1 + \frac{1}{T_i}\right)$$

Теперь вспомним, чему равно K :

$$K = \frac{1}{2 \ln(1 + 1/T_i)}$$

$$2 \cdot \left(\frac{1}{2 \ln(1 + 1/T_i)}\right) \cdot \ln\left(1 + \frac{1}{T_i}\right) = \frac{2 \ln(1 + 1/T_i)}{2 \ln(1 + 1/T_i)} = 1$$

Очевидно что подынтегральная функция неотрицательна, значит она действительно задаёт плотность.

Тогда

$$G_T(y) = \prod_{i=1}^D g_{(i;T)}(y_i)$$

Пусть у нас есть случайное число $\alpha_i \in [0, 1]$, тогда мы хотим найти такую координату z_i , что

$$F(z_i) = \int_{-1}^{z_i} g_{(i;T)}(y) dy = \alpha_i,$$

то есть

$$z_i = F^{-1}(\alpha_i)$$

Давайте решим это уравнение на z_i :

Так как распределение симметрично относительно нуля, давайте рассмотрим случай $z_i > 0$ (прыжок вправо). Тогда интеграл от -1 до z_i можно разбить на две части: от -1 до 0 и от 0 до z_i .

Весь интеграл по $[0, 1]$ равен 0.5 (половина общей площади).

$$\int_{-1}^0 g(y) dy = 0.5$$

$$\int_0^{z_i} g(y) dy = \alpha_i - 0.5$$

(Если $\alpha_i < 0.5$, мы будем интегрировать в отрицательную область, отсюда и берется функция $\text{sign}(\alpha_i - 0.5)$).

$$\int_0^{z_i} \frac{K}{|y| + T_i} dy = \alpha_i - 0.5$$

где $K = \frac{1}{2 \ln(1+1/T_i)}$.

$$K \cdot \ln(y + T_i) \Big|_0^{z_i} = \alpha_i - 0.5$$

$$K \cdot (\ln(z_i + T_i) - \ln(T_i)) = \alpha_i - 0.5$$

$$K \cdot \ln\left(\frac{z_i + T_i}{T_i}\right) = \alpha_i - 0.5$$

$$\ln \left(1 + \frac{z_i}{T_i} \right) = \frac{\alpha_i - 0.5}{K}$$

Вспомним, что $\frac{1}{K} = 2 \ln(1 + \frac{1}{T_i})$:

$$\ln \left(1 + \frac{z_i}{T_i} \right) = (\alpha_i - 0.5) \cdot 2 \ln \left(1 + \frac{1}{T_i} \right)$$

$$\ln \left(1 + \frac{z_i}{T_i} \right) = (2\alpha_i - 1) \cdot \ln \left(1 + \frac{1}{T_i} \right)$$

$$1 + \frac{z_i}{T_i} = \left(1 + \frac{1}{T_i} \right)^{2\alpha_i - 1}$$

$$\frac{z_i}{T_i} = \left(1 + \frac{1}{T_i} \right)^{2\alpha_i - 1} - 1$$

$$z_i = T_i \left(\left(1 + \frac{1}{T_i} \right)^{2\alpha_i - 1} - 1 \right)$$

Мы решали для $z_i > 0$. Если $\alpha_i < 0.5$, то результат будет отрицательным. Чтобы учесть оба случая, добавляем $\text{sign}(\alpha_i - 0.5)$:

$$z_i = \text{sign}(\alpha_i - 0.5) \cdot T_i \left(\left(1 + \frac{1}{T_i} \right)^{|2\alpha_i - 1|} - 1 \right)$$

Тогда новое состояние системы x_i определяется по координатно следующим образом: $x'_i = x_i + z_i(B_i - A_i)$.

Можно показать, что при $T_i(k) = T_{(i;0)} e^{-c_i k^{\frac{1}{D}}}$ алгоритм гарантирует нахождение глобального минимума. Давайте идейно покажем это:

Чтобы алгоритм нашел глобальный минимум, сумма вероятностей совершить прыжок на нужное расстояние из любой неоптимальной области должна быть бесконечной.

$$P_k \approx \int_L^\infty G_{T(k)}(y) dy \approx \frac{1}{\ln(1 + 1/T(k))}$$

Подставим закон охлаждения $T(k) = T_0 \exp(-ck^{1/D})$:

$$P_k \approx \frac{1}{\ln(1 + \frac{1}{T_0 \exp(-ck^{1/D})})} \approx \frac{1}{\ln(\exp(ck^{1/D}))} = \frac{1}{c \cdot k^{1/D}}$$

$$\sum_{k=1}^{\infty} P_k \approx \sum_{k=1}^{\infty} \frac{1}{c \cdot k^{1/D}}$$

Если $D \geq 1$, ряд $\sum \frac{1}{k^{\frac{1}{D}}}$ (где $\frac{1}{D} \leq 1$) — это обобщенный гармонический ряд, который расходится.

Обычно c_i задаётся как $m_i e^{-\frac{n_i}{D}}$, где параметры n_i и m_i подбираются.

В отличие от логарифмического убывания в классическом отжиге, VFSA задает экспоненциальное убывание температуры, что позволяет алгоритму значительно быстрее сходиться к области глобального минимума, существенно экономя вычислительные ресурсы. Введение индивидуальной температуры для каждой координаты позволяет алгоритму адаптироваться к конкретной задаче. Если параметры имеют разную степень влияния на целевую функцию, VFSA автоматически настраивает скорость поиска по каждой координате.

Метод также допускает эффективное моделирование распределения независимо от размерности пространства состояний, что делает его предпочтительным при решении высокоразмерных задач оптимизации.

Тем не менее, метод обладает и определенным недостатком, который ограничивает его повсеместное применение: Высокая эффективность VFSA напрямую зависит от точного подбора управляющих параметров m_i , n_i .

4. Тушение

В реальных прикладных задачах поиск строгого глобального минимума зачастую ограничен дефицитом вычислительных ресурсов. Во многих случаях для достижения поставленной цели достаточно найти не глобальный оптимум, а решение, находящееся в достаточно малой окрестности к оптимальному. Данная необходимость привела к появлению методов так называемого тушения (simulated quenching). Основная концепция этих методов заключается в модификации классических схем оптимизации путем ускорения процесса охлаждения системы. Методологически «тушение» строится на двух ключевых изменениях:

1. **Использование апробированных семейств распределений:** В качестве основы берутся распределения, характерные для классических алгоритмов (например, нормальное распределение из Больцмановского отжига).
2. **Ускоренный закон убывания температуры:** Вместо теоретически обоснованных медленных законов (логарифмических), применяется значительно более интенсивное снижение температуры.

Примером реализации такой схемы служит геометрическое охлаждение:

$$T_{k+1} = c \cdot T_k, \quad \text{где } c \in (0.7; 0.9)$$

Хотя подобные методы не гарантируют сходимости к глобальному минимуму, на практике они демонстрируют высокую эффективность.

Обычно температура убывает по экспоненте, поэтому для известных распределений алгоритм сходиться не будет. Давайте докажем полезное утверждение:

Для любого $\epsilon > 0$ существует такое число итераций k , что вероятность нахождения в ϵ -окрестности глобального минимума $P(f(x_k) - f(x^*) < \epsilon)$ больше, чем $1 - \delta$ при некотором $\delta \in (0, 1)$.

Доказательство: Определим S_ϵ как ϵ -окрестность глобального минимума:

$$S_\epsilon = \{x \in S \mid f(x) - f(x^*) < \epsilon\}$$

Введем понятие бассейна притяжения $B(S_\epsilon)$ — это множество таких точек пространства S , из которых любой строго монотонный локальный спуск (где принимаются только шаги $\Delta E < 0$) гарантированно приводит в S_ϵ . Поскольку начальная температура $T_0 > 0$, а процесс блуждания случаен, существует строго положительная вероятность $p > 0$, что на некоторой итерации N алгоритм окажется внутри бассейна притяжения:

$$P(x_N \in B(S_\epsilon)) = p > 0$$

Для итераций $k \gg N$ температура $T_k \rightarrow 0$ (в силу $c^k \rightarrow 0$ при $c < 1$). Вероятность принятия ухудшающего шага стремится к нулю:

$$\lim_{k \rightarrow \infty} \exp\left(-\frac{\Delta E}{T_k}\right) = 0, \quad \text{для } \Delta E > 0$$

С этого момента алгоритм вырождается в градиентный спуск. Если система к моменту N находилась внутри бассейна притяжения $x_N \in B(S_\epsilon)$, то все последующие итерации будут лишь стягивать решение ко дну этого бассейна, пока оно не достигнет S_ϵ .

Вероятность того, что к итерации k алгоритм достигнет ϵ -окрестности оптимума, ограничена снизу вероятностью попадания в правильный бассейн притяжения до момента полного остывания системы:

$$P(f(x_k) - f(x^*) < \epsilon) \geq P(x_N \in B(S_\epsilon)) = p$$

Положив $p = 1 - \delta$, получаем требуемое утверждение.

Наличие константы δ в доказательстве означает вероятность провала (алгоритм остался в локальном минимуме и $x_N \notin B(S_\epsilon)$). Однако это легко компенсируется перезапусками. Если мы запустим алгоритм быстрого тушения независимо M раз из разных начальных точек, вероятность того, что хотя бы один запуск попадет в ϵ -окрестность глобального оптимума, составит $1 - \delta^M$.

Так как $\delta < 1$, при увеличении числа перезапусков M величина $\delta^M \rightarrow 0$, а вероятность успеха стремится к 1, поэтому на практике выгоднее сделать 10 быстрых циклов тушения, чем 1 долгий цикл классического отжига.

5. Масштабирование

В многомерных задачах целевая функция может быть очень "крутой" по одной координате (частная производная очень большой) и очень "пологой" по другой (маленькой). Если понижать температуру одинаково для всех измерений,

алгоритм может застрять. В пологих направлениях шаги станут слишком маленькими (при маленькой температуре вероятность выбрать $y_i \approx 0$ растёт), и решение перестанет двигаться по какой-то из координат. Поэтому хочется уметь менять температуру для каждой координаты отдельно (отжиг-то сойдётся, однако при $D = 100$ это произойдёт за слишком большое время).

Пусть s_i это оценка частной производной целевой функции по некоторой координате в точке текущего решения:

$$s_i \geq \left| \frac{\partial f}{\partial x_i}(x_i^{(min)}) \right| \text{ и } s_{max} = \max_{1 \leq i \leq D} s_i$$

Для соответствующего измерения будем менять температуру следующим образом:

$$T_i(k) = T(k) \frac{s_{max}}{s_i}$$

Интуиция выбора именно такого коэффициента в следующем:

Понятно, что $|\Delta E_i| \approx s_i \cdot |\Delta x_i|$, при этом $E(\Delta x_i) \sim T_i$, то есть $E(|\Delta E_i|) \sim s_i \cdot T_i$. Чтобы алгоритм работал эффективно, нам нужно, чтобы масштаб изменения энергии был примерно одинаковым, в какую бы сторону мы ни шагнули. Для этого нужно потребовать, чтобы произведение крутизны на температуру было константой для всех координат. Отсюда $s_i T_i = s_{max} T(k)$.

Сложность может возникнуть при подсчёте градиента функции, однако, если градиент в точке вообще считается, то это происходит не более чем в 3 раза медленнее значения самой функции (автоград).

6. Критерии останова

6.1. Температурный критерий

Этот критерий опирается на физический смысл абсолютного нуля температур. Задаётся нижняя граница температуры $T_{min} > 0$ (обычно порядка $10^{-5} \dots 10^{-8}$). Алгоритм останавливается на итерации k , если выполняется условие:

$$T(k) \leq T_{min}$$

Рассмотрим функцию вероятности принятия нового состояния $h(\Delta E, T) = \exp(-\frac{\Delta E}{T})$. При вычислении предела для любого ухудшающего шага ($\Delta E > 0$):

$$\lim_{T \rightarrow +0} \exp\left(-\frac{\Delta E}{T}\right) = 0$$

Следовательно, при $T \leq T_{min}$ вероятность перехода в состояние с большей энергией становится околонулевой. Алгоритм полностью теряет способность преодолевать локальные минимумы. Кроме того, порождающее семейство $G(x, T)$ при малых T начинает генерировать точки x' в бесконечно малой ε -окрестности текущего состояния x . В этой фазе метод отжига фактически вырождается в жадный локальный спуск, и дальнейшее выполнение дорогостоящих стохастических итераций нецелесообразно.

6.2. Критерий стагнации

Алгоритм прекращает работу, если лучшее найденное значение целевой функции не обновляется на протяжении заданного промежутка времени длиной N_{stag} итераций.

Пусть $E_{min}^{(k)}$ — значение глобального минимума, найденное к k -й итерации: $E_{min}^{(k)} = \min_{j \leq k} f(x^{(j)})$. Критерий срабатывает, если для заданного допуска $\varepsilon \geq 0$:

$$|E_{min}^{(k)} - E_{min}^{(k-N_{stag})}| \leq \varepsilon$$

Выполнение этого условия означает, что алгоритм застрял. Если текущая температура $T(k)$ уже достаточно мала, то вероятность $P(|\Delta x| \geq L)$ сгенерировать прыжок достаточной длины для выхода из этого бассейна становится околонулевой.

6.3. Acceptance Rate

В физике твердого тела процесс отжига заканчивается кристаллизацией — состоянием, когда атомы фиксируются в узлах кристаллической решетки и перестают перемещаться. В алгоритме это отслеживается через эмпирическую вероятность принятия нового состояния.

Пусть вероятность принятия нового состояния $P_{acc}(x, T)$ при текущей температуре T и состоянии x . Для непрерывного пространства состояний S эта вероятность выражается через интеграл по плотности порождающего распределения $g(x'; x, T)$:

$$P_{acc}(x, T) = \int_S g(x'; x, T) \cdot h(\Delta E, T) dx' = \int_S g(x'; x, T) \cdot \min \left(1, \exp \left(-\frac{f(x') - f(x)}{T} \right) \right) dx'$$

Разобьем пространство S на две области относительно текущей точки x : область улучшающих решений $S^- = \{x' \mid f(x') \leq f(x)\}$ и область ухудшающих решений $S^+ = \{x' \mid f(x') > f(x)\}$.

Тогда интеграл распадается на две части:

$$P_{acc}(x, T) = \int_{S^-} g(x'; x, T) dx' + \int_{S^+} g(x'; x, T) \exp \left(-\frac{\Delta E}{T} \right) dx'$$

Если алгоритм находится в ε -окрестности строгого локального (или глобального) минимума, мера Лебега множества S^- стремится к нулю, следовательно, первый интеграл $\rightarrow 0$.

Вычислим

$$\lim_{T \rightarrow 0} \int_{S^+} g(x'; x, T) \exp \left(-\frac{f(x') - f(x)}{T} \right) dx'$$

Зафиксируем любую точку $x' \in S^+$. Для этой конкретной точки разность строго положительна: $f(x') - f(x) = c_{x'} > 0$. Тогда при $T \rightarrow 0$:

$$\lim_{T \rightarrow 0} \exp \left(-\frac{c_{x'}}{T} \right) = 0$$

Следовательно, подынтегральная функция сходится к нулю поточечно (почти всюду на S^+). Так как экспонента всегда ≤ 1 :

$$g(x'; x, T) \exp \left(-\frac{f(x') - f(x)}{T} \right) \leq g(x'; x, T)$$

Поскольку $g(x'; x, T)$ — это плотность вероятности, то она интегрируема, значит условие теоремы Лебега выполнено. Мы имеем право поменять местами предел и интеграл:

$$\int_{S^+} \lim_{T \rightarrow 0} \left[g(x'; x, T) \exp \left(-\frac{f(x') - f(x)}{T} \right) \right] dx' = \int_{S^+} 0 dx' = 0$$

Таким образом, $\lim_{T \rightarrow 0} P_{acc}(x, T) = 0$.

1. На каждой итерации j алгоритм генерирует шаг и решает: принять его или отвергнуть. Мы вводим индикаторную случайную величину:

$$I_j = \begin{cases} 1, & \text{шаг принят} \\ 0, & \text{шаг отвергнут} \end{cases}$$

2. Выбирается размер скользящего окна M . 3. $\rho(k)$ вычисляется как среднее арифметическое индикаторов в этом окне:

$$\rho(k) = \frac{1}{M} \sum_{j=k-M+1}^k I_j$$

Согласно закону больших чисел, эмпирическая частота события при достаточно большом M сходится по вероятности к матожиданию этого события:

$$\rho(k) \xrightarrow{P} \mathbb{E}[I] = P_{acc}(x_k, T_k)$$

Условие $\rho(k) \leq \rho_{min}$ является индикатором того, что процесс блуждания локализован на дне бассейна притяжения оптимума, где почти любое сгенерированное изменение x' приводит к существенному росту целевой функции ($\Delta E \gg T$) и отвергается отжигом.

7. Функция Растригина

Для $x \in \mathbb{R}^n$ функция Растригина это

$$f(x) = An + \sum_{i=1}^n [x_i^2 - A \cos(2\pi x_i)]$$

Функция является невыпуклой и мультимодальной, поэтому на ней часто испытывают алгоритмы оптимизации. Давайте с помощью VFSA и эвристики тушения найдем глобальный минимум. Глобальным минимумом очевидно является $x^* = (0, 0, \dots, 0)$.

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  const long double pi = acos(-1.0);
6
7  long double func(long double A, vector<long double>& x) {
8      long double res = 0;
9      for (int i = 0; i < (int) x.size(); i++) {
10         res += A + x[i] * x[i] - A * cos(2 * pi * x[i]);
11     }
12     return res;
13 }
14
15 int main() {
16     ios_base::sync_with_stdio(false);
17     cin.tie(nullptr);
18     random_device rd;
19     mt19937_64 gen(rd());
20     uniform_real_distribution<long double> dist(0.0, 1.0);
21
22     int n;
23     long double A_const;
24     cin >> n;
25     cin >> A_const;
26     vector<long double> x_curr(n, 10);
27     vector<long double> x(n);
28     vector<long double> temp(n, 100);
29     long double A = -(A_const + 101);
30     long double B = (A_const + 101);
31     long double c = 0.9997;
32     long double norm = 1;
33     long double E_curr = func(A_const, x_curr);
34     long double E;
35     while (norm > 1e-14) {
36         norm = 0;
37         for (int i = 0; i < n; i++) {
38             long double alpha = dist(gen);
39             int flag = 1;
40             if (alpha < 0.5) {
41                 flag = -1;
42             }
43             long double z = flag * temp[i] * (pow((1.0 + 1.0 / temp[i]), abs(2 * alpha -
44                 ↪ 1)) - 1);
45             x[i] = x_curr[i] + z * (B - A);
46             if (x[i] < A) {
47                 x[i] = A;
48             }
49             if (x[i] > B) {
50                 x[i] = B;
51             }
52             temp[i] = c * temp[i];
53         }
54         E = func(A_const, x);
55         long double delta = E - E_curr;
```

```

55         if (delta < 1e-10) {
56             x_curr = x;
57             E_curr = E;
58         } else {
59             long double p = exp(-delta / norm);
60             long double chance = dist(gen);
61
62             if (chance < p) {
63                 x_curr = x;
64                 E_curr = E;
65             }
66         }
67         for (int i = 0; i < n; i++) {
68             norm += temp[i] * temp[i];
69         }
70         norm = sqrt(norm);
71     }
72
73     for (long double i : x_curr) {
74         cout << i << " ";
75     }
76
77     return 0;
78 }

```

Программа отработала 10 раз для $n = 15$ и $A = 1$ при коэффициенте охлаждения $c = 0.9997$. В результате каждой отработки выводился вектор-ответ и невязка:

0.000832234 0.00114915 0.00159381 -0.00620998 -0.00643665 0.00631407 0.00174351 0.00611177 0.00483442 0.000456229
0.0197043 0.00141412 -0.00312761 0.00914008 0.0108235

0.0280912

-0.014562 -0.00291253 -0.00943189 0.00182885 0.0025382 -0.00474213 -0.00791689 0.00489003 -0.00655465 0.0102933 0.00318911
0.00138086 0.00585304 0.00282713 -0.00341653

0.0253629

-0.00994107 -0.0109075 -0.00445469 -0.00320247 0.00231641 0.00086237 -0.00334763 -0.0043189 0.00649391 0.00289397 -
0.0111413 0.000557277 0.0040977 0.00233657 0.013148

0.0255729

0.000382307 0.00328691 -0.001561 0.000474607 0.0103831 -0.00318532 -0.000425536 -0.00306108 -0.000637293 0.00341917
-0.00577536 0.00647445 0.0014717 0.00193006 0.00738891

0.0169993

0.00136719 -0.00258338 0.00889818 0.000729063 0.00251718 -0.00309927 0.000856601 -0.00103928 -0.0043774 0.00217436 -
0.00254386 -0.00462959 -0.00404708 -0.0044988 -0.00624367

0.0152775

-0.0164777 0.00133256 0.00180071 0.000866053 -0.00102595 -0.00204969 0.00150813 0.00315185 -0.0301701 0.000373814 0.000621922
0.00677604 -0.00052383 -0.00431742 9.27396e-05

0.0356419

0.00200592 0.0108808 -0.00114715 0.00153217 -0.00703915 -0.000841326 0.00183882 0.000569468 -0.000747596 0.00348064
-0.00235284 0.00266595 0.000862863 -0.00578028 -0.00178691

0.0155791

0.0085281 0.00497782 0.0072449 0.00768768 0.00272336 -0.00390436 0.00841066 -0.00234084 0.00565457 0.00698118 0.00175181
-0.00321177 0.00218354 0.00717679 1.25143e-05

0.021409

0.0191431 0.00544331 -0.00164624 -0.00755424 0.00772467 0.000606859 0.00352026 0.000321034 0.00115074 0.00596508 -
0.0020485 0.0031837 0.00387968 0.00552447 -0.00482191

0.0254648

4.56303e-05 0.000250563 -0.0091648 -0.00157606 0.00327582 -0.00195233 -0.00232812 0.00162392 0.00366631 1.19048e-05
0.00255008 -0.00661178 -0.00264398 -0.00324745 0.00530412

0.0147808

Среднее значение невязки составило 0.02241794, время каждого теста - 216452875 ns. Значение невязки свидетельствует о том, что алгоритм не застрял в локальном минимуме, поскольку при $A = 1$ наименьший по модулю экстремум имеет координату-абсциссу $x > 0.95$.